

SRS Document

Functional requirements

FR-01 | File selection

The system shall allow the user to select a BasicML text file from any accessible location through a graphical file picker.

FR-02 | Program loading

The system shall load the words of a valid BasicML file into consecutive memory locations beginning at address 00.

FR-03 | READ input

The system shall request and accept a signed integer from -9999 through 9999 through the GUI when executing READ and store the value at the specified memory address.

FR-04 | Input validation and retry

The system shall reject blank, non-integer, or out-of-range READ input and preserve the pending READ instruction and prior execution state while the user corrects and resubmits the value without restarting execution.

FR-05 | WRITE output

The system shall display each WRITE value in the GUI output area using four magnitude digits and a minus sign for negative values.

FR-06 | Addition

The system shall add the operand memory value to the accumulator when executing ADD.

FR-07 | Subtraction

The system shall subtract the operand memory value from the accumulator when executing SUBTRACT.

FR-08 | Multiplication

The system shall multiply the accumulator by the operand memory value when executing MULTIPLY.

FR-09 | Division

The system shall divide the accumulator by a nonzero operand memory value using integer division that truncates the quotient toward zero.

FR-10 | Unconditional branch

The system shall set the instruction counter to the operand address when executing BRANCH.

FR-11 | Negative branch

The system shall branch to the operand address on BRANCHNEG only when the accumulator is negative; otherwise execution shall continue sequentially.

FR-12 | Zero branch

The system shall branch to the operand address on BRANCHZERO only when the accumulator equals zero; otherwise execution shall continue sequentially.

FR-13 | Halt

The system shall stop executing instructions when it encounters HALT.

FR-14 | Arithmetic overflow

The system shall retain only the lowest four magnitude digits of an overflowing arithmetic result, preserving its sign, as the accumulator value used by subsequent instructions.

FR-15 | File-error recovery

The system shall display an error message for an invalid or malformed program file and allow the user to select another file without restarting the application.

FR-16 | Instruction validation

The system shall detect instructions containing invalid operation codes.

FR-17 | Code entry and viewing

The GUI shall provide an area for users to enter or view BasicML code.

FR-18 | Program execution control

The GUI shall provide a control for executing the loaded BasicML program.

FR-19 | Accumulator display

The GUI shall display the current value of the accumulator.

FR-20 | Instruction display

The GUI shall display the current instruction being executed.

FR-21 | Execution status

The GUI shall indicate when program execution has completed.

FR-22 | Error display

The GUI shall display an error message when an error occurs.

FR-23 | Step execution

The system shall allow the user to execute a program one instruction at a time.

FR-24 | Execution continuation

The system shall allow the user to continue execution after stepping through instructions.

FR-25 | File validation

The system shall accept properly formatted four-digit BasicML words, ignore blank lines, reject malformed words, and enforce the maximum program size when validating a selected file.

FR-26 | Machine initialization

The system shall reset the machine state before execution of a newly loaded program begins.

FR-27 | Stop execution

The system shall allow the user to stop a BasicML program while it is executing.

FR-28 | LOAD

The system shall copy the value from the specified memory address into the accumulator when executing LOAD.

FR-29 | STORE

The system shall copy the accumulator value into the specified memory address, replacing its previous contents, when executing STORE.

FR-30 | Output retention

The system shall preserve displayed program output after execution stops, completes, or encounters an error and clear the previous output when a new program is successfully loaded.

Non-functional requirements

NFR-01 | Responsiveness

The system shall remain responsive to GUI controls, including Stop, during program execution and yield to the GUI event loop after no more than 100 instructions in a single scheduled execution batch.

NFR-02 | Modularity

The system shall separate user-interface code from the simulator data model and business logic using separate classes or modules, keeping simulator and controller logic independent of Tkinter imports and GUI widget objects.

NFR-03 | Dependencies

The system shall run using Python 3.10 or newer with its standard library and Tcl/Tk support, without requiring third-party Python packages.

NFR-04 | Resource usage

The simulator shall run within reasonable CPU and memory usage on a typical computer.

NFR-05 | Recoverability

The system shall allow the user to reset the simulator to a known initial state after an error or completed execution.

NFR-06 | Execution response time

The system shall begin executing a BasicML program within 2 seconds of the user selecting the execution control.

NFR-07 | Usability

The system shall provide clearly labeled GUI controls, prompts, input fields, and output areas so users can understand their purpose.